

Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 6ο : Συστήματα Μικροϋπολογιστών

Δημήτριος Θανόγιαννης – 03121902

1η Ομάδα Ασκήσεων

Ασκήσεις προσομοίωσης:

1η Άσκηση:

α) Ο κώδικας που αποθηκεύει τους αριθμούς 0-255 στις θέσεις μνήμης 0200 H – 02FF H είναι ο ακόλουθος:

```
.include "m16def.inc"

.def count = r18
.def temp = r17
.def zero_low = r14
.def zero_high = r15
.def between_count = r16
.def i = r19

.org 0x0000
    rjmp start

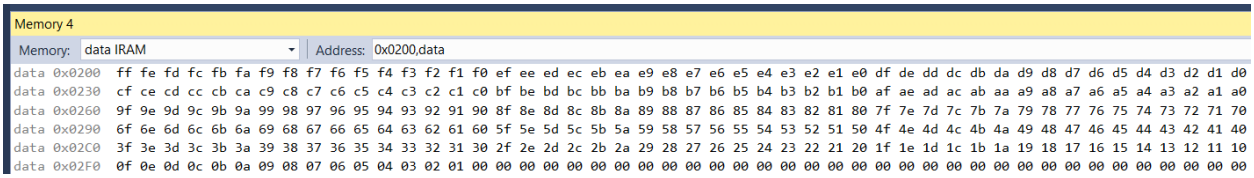
.org 0x002A ;start the program after the dedicated memory space for interrupts

start:
    ;stack pointer initialization
    ldi temp, low(RAMEND)
    out SPL, temp
    ldi temp, high(RAMEND)
    out SPH, temp

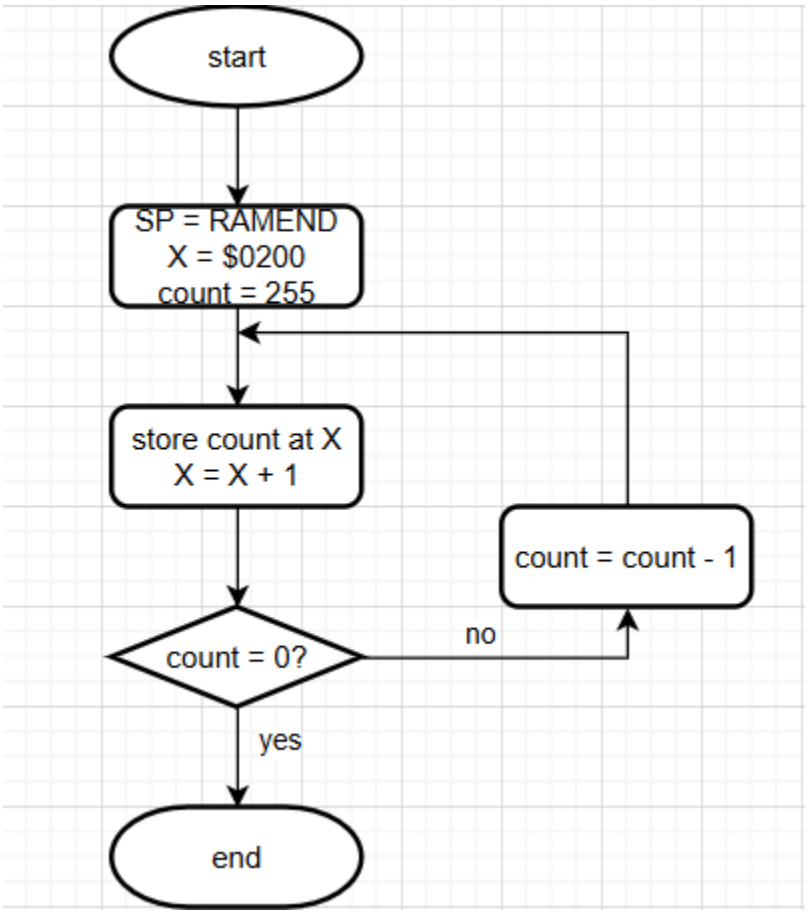
    clr r26 ;xl = 00
    ldi r27, $02 ;xh = 20 => start = $0200
    ldi count, 255 ;numbers 255 -> 0

loop_a:
    st x+, count ;post-increment memory addressing
    tst count ;check for zero counter before decrement, to avoid overflow and include 0
    breq loop_b ;if zero, exit
    dec count
    rjmp loop_a
```

Πρόκειται για το πρώτο μέρος κώδικα που εκτελεί όλα τα ερωτήματα της 1ης άσκησης, το ένα μετά το άλλο. Η μνήμη δεδομένων μετά την εκτέλεση του α) ερωτήματος έχει την ακόλουθη μορφή:



Όπως παρατηρούμε, η μνήμη δεδομένων έχει την αναμενόμενη μορφή. Το διάγραμμα ροής του α) ερωτήματος είναι το ακόλουθο:



β) Ο κώδικας που μετρά το πλήθος των μηδενικών στους προαναφερθείς αριθμούς (0-255) και αποθηκεύει το αποτέλεσμα στο διπλό καταχωρητή r15:r14 είναι ο ακόλουθος. Πρόκειται για συνέχεια του κώδικα του α) ερωτήματος.

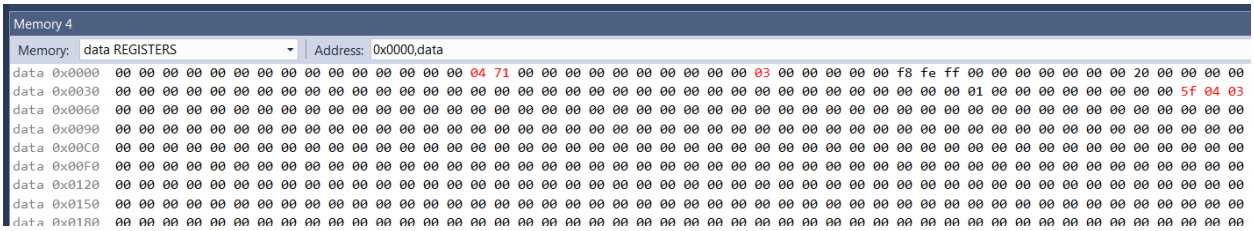
```
loop_b:
    ld temp, -x ;load the memory addresses by decrementing, the opposite way
    ldi i, 8 ;intialize i with 8, used for bits in a byte
    rjmp zero_counting

inc_loop:
    inc count
    cpi count, 255 ;check for all numbers
    breq loop_c
    rjmp loop_b

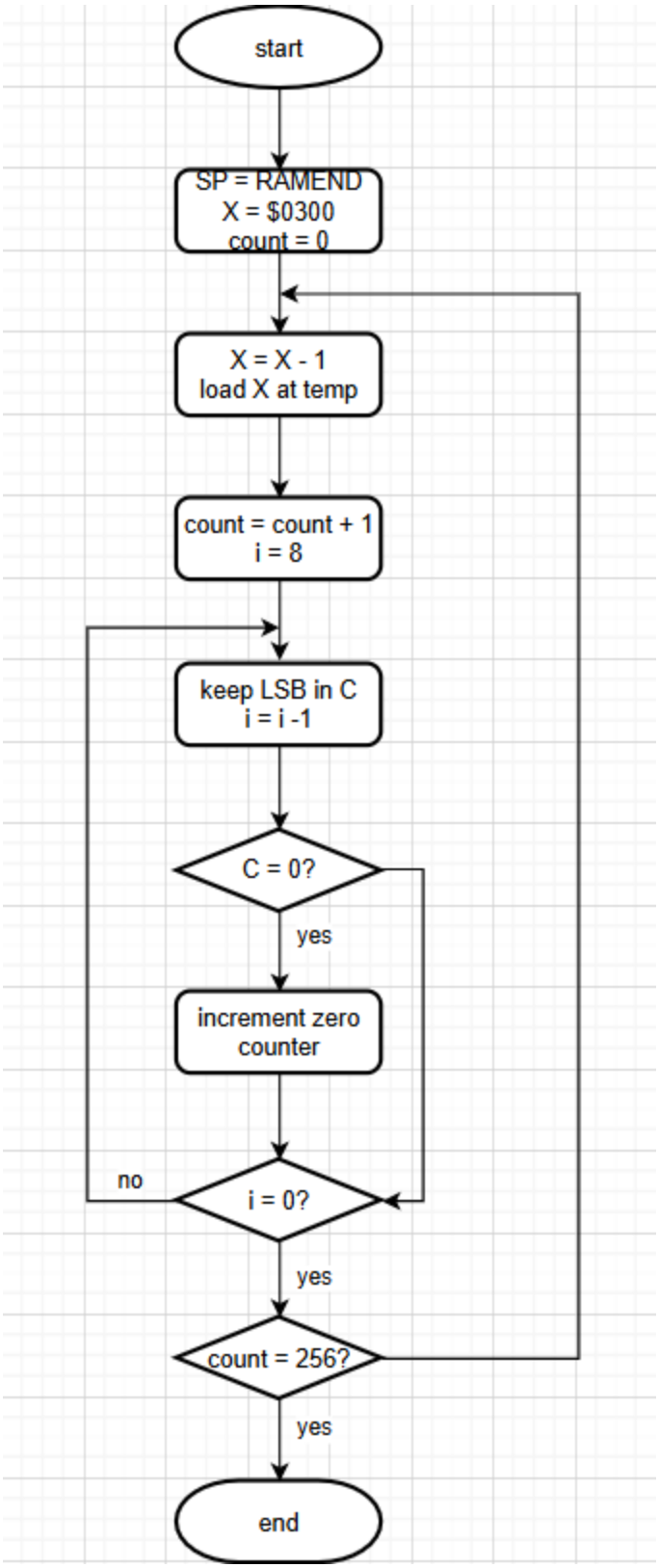
zero_counting:
    lsr temp ;put the LSB to the carry bit
    brcs dec_loop ;if it is not zero, continue
    inc zero_low ;if it is zero, add to the sum
    brne dec_loop ;if there is no overflow in the low register (zero flag is 0, because 255 + 1 = 0), continue
    inc zero_high ;else increment the high register
    rjmp dec_loop

dec_loop:
    dec i
    breq inc_loop ;if all bits are checked, next Byte
    rjmp zero_counting
```

Η μνήμη καταχωρητών μετά την εκτέλεση ολόκληρου του κώδικα έχει την ακόλουθη αναμενόμενη μορφή. Το σημείο ενδιαφέροντος εδώ είναι το αναμενόμενο 04:00 (1024 δυαδικό, εφόσον 2048 είναι συνολικά τα ψηφία από το 0-255, τα μισά θα είναι 0) στους καταχωρητές r15:r14.



Το διάγραμμα ροής του β) ερωτήματος είναι το ακόλουθο:


start
SP = RAMEND
X = $0300
count = 0
X = X - 1
load X at temp
count = count + 1
i = 8
keep LSB in C
i = i -1
C = 0?
yes
increment zero
counter
no
i = 0?
yes
count = 256?
yes
end

γ) Ο κώδικας που υπολογίζει το πλήθος των αριθμών από το 0-255 που είναι στο διάστημα 10 H <= x <= 80 H ([16-128] δεκαδικό) είναι ο ακόλουθος. Πρόκειται για συνέχεια του κώδικα του β) ερωτήματος.

```
loop_c:
    clr between_count ;initialization of counter for numbers between 16 and 128
    clr r26 ;xl = 00
    ldi r27, $02 ;xh = 20 => start = $0200
    clr count ;so that we go from 0 to 255 and to 0 again, executing 256 iterations

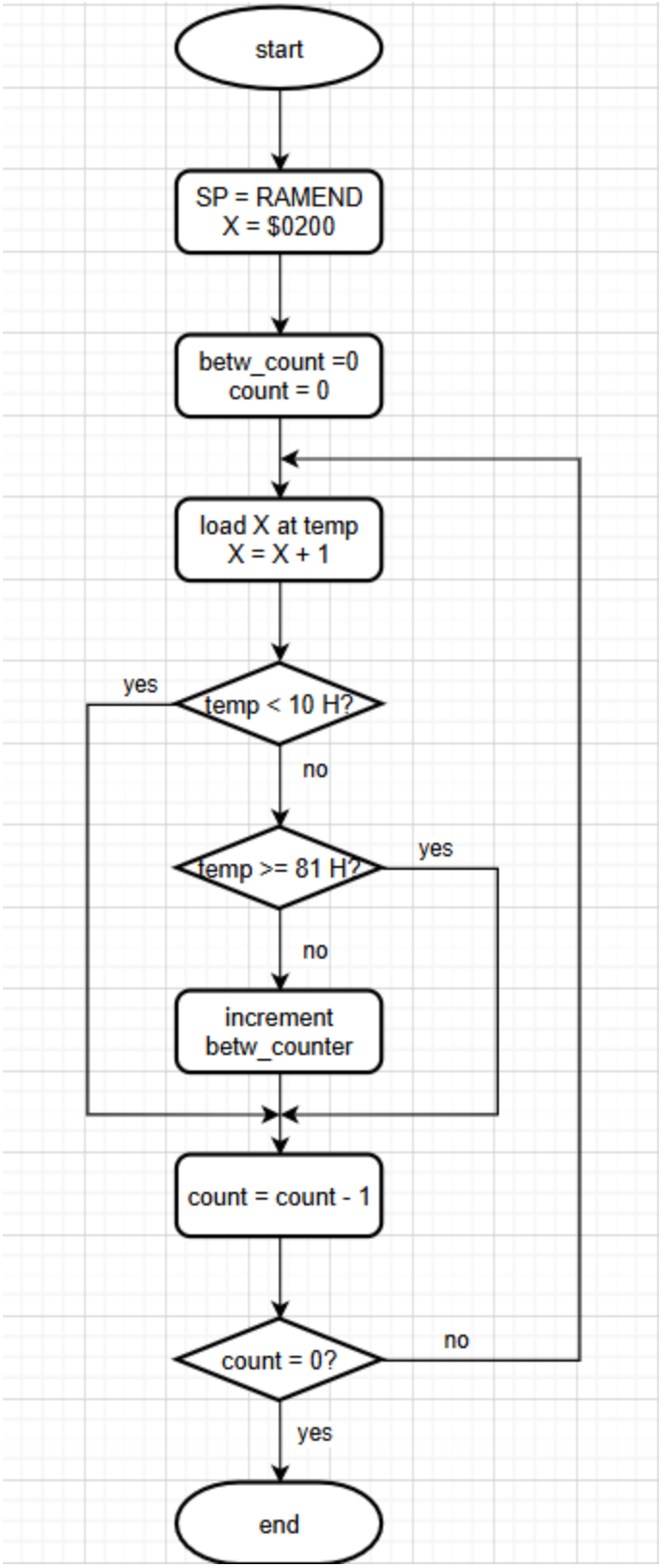
compare_loop:
    ld temp, x+ ;load numbers
    cpi temp, $10
    brlo skip_number ;discard if < 10
    cpi temp, $81
    brsh skip_number ;discard if >= 81
    inc between_count ;else include number

skip_number:
    dec count ;counter was at 255 from b)
    brne compare_loop ;this includes the iteration when counter is 0

end:
    rjmp end
```

Το σημείο ενδιαφέροντος στον πίνακα των καταχωρητών για το ερώτημα γ) είναι το αναμενόμενο 71 (113 δεκαδικό = 128 – 16 + 1) στον καταχωρητή r16.

Το διάγραμμα ροής του γ) ερωτήματος είναι το ακόλουθο:


start
SP = RAMEND
X = $0200
betw_count =0
count = 0
load X at temp
X = X + 1
yes
temp < 10 H?
no
temp >= 81 H?
yes
no
increment
betw_counter
count = count - 1
count = 0?
no
yes
end

2η Άσκηση:

Ο κώδικας που μετράει το πλήθος των εναλλαγών των bits της εισόδου (PORTB) και τις αποτυπώνει σε unary μορφή στα MSB της εξόδου (PORTD) είναι ο ακόλουθος:

```
.include "m16def.inc"

.def temp = r16
.def prev_bit = r17
.def comp = r18
.def i = r19
.def ans = r20

.org 0x0000
    rjmp start

.org 0x002A ;start the program after the dedicated memory space for interrupts

start:
    ;stack pointer initialization
    ldi temp, low(RAMEND)
    out SPL, temp
    ldi temp, high(RAMEND)
    out SPH, temp

    ;I/O initialization
    clr temp
    out DDRB, temp
    ser temp
    out PORTB, temp ;pull-up resstance activation
    out DDRD, temp

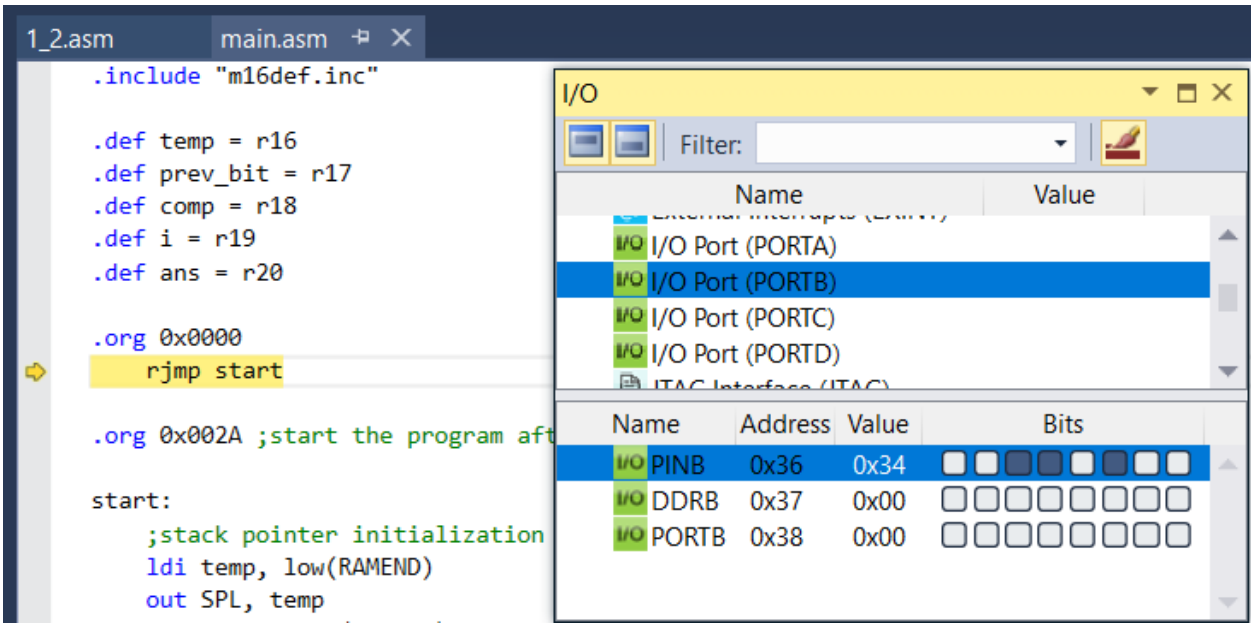
init:
    in temp, PINB
    mov prev_bit, temp
    andi prev_bit, 0b00000001 ;initialize register with LSB for comparison
    clr ans ;initialize answer with 0
    ldi i, 7 ;initialize i for 7 iterations
```

```
loop:
    bst temp, 0 ;store LSB in T
    lsr temp ;shift right for next bit
    bld prev_bit, 0 ;load LSB from T
    mov comp, temp ;copy the input to compare register
    andi comp, 0b00000001 ;keep only LSB
    eor comp, prev_bit ;XOR to check for change, 1 = change, 0 = no change
    cpi comp, 0
    breq inc_loop ;if there is no change, next iteration
    sec ;set carry bit to add to the MSB
    ror ans ;if there is a change, add another one

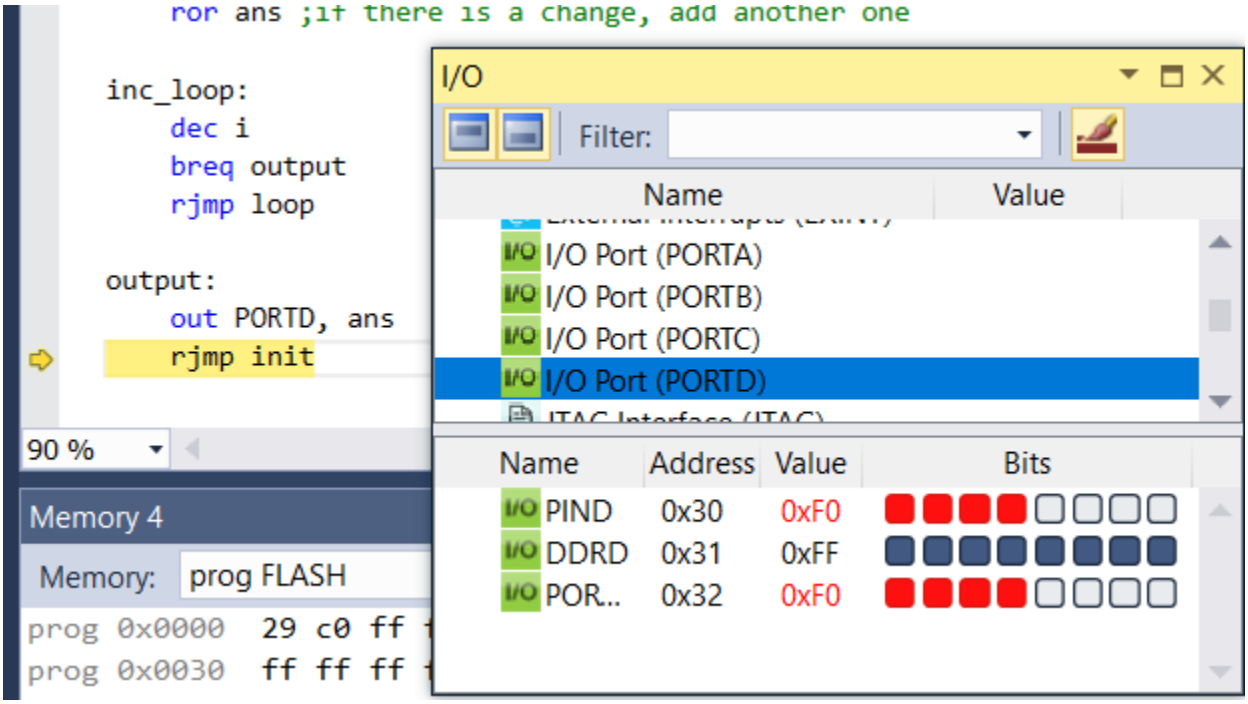
inc_loop:
    dec i
    breq output
    rjmp loop

output:
    out PORTD, ans
    rjmp init
```

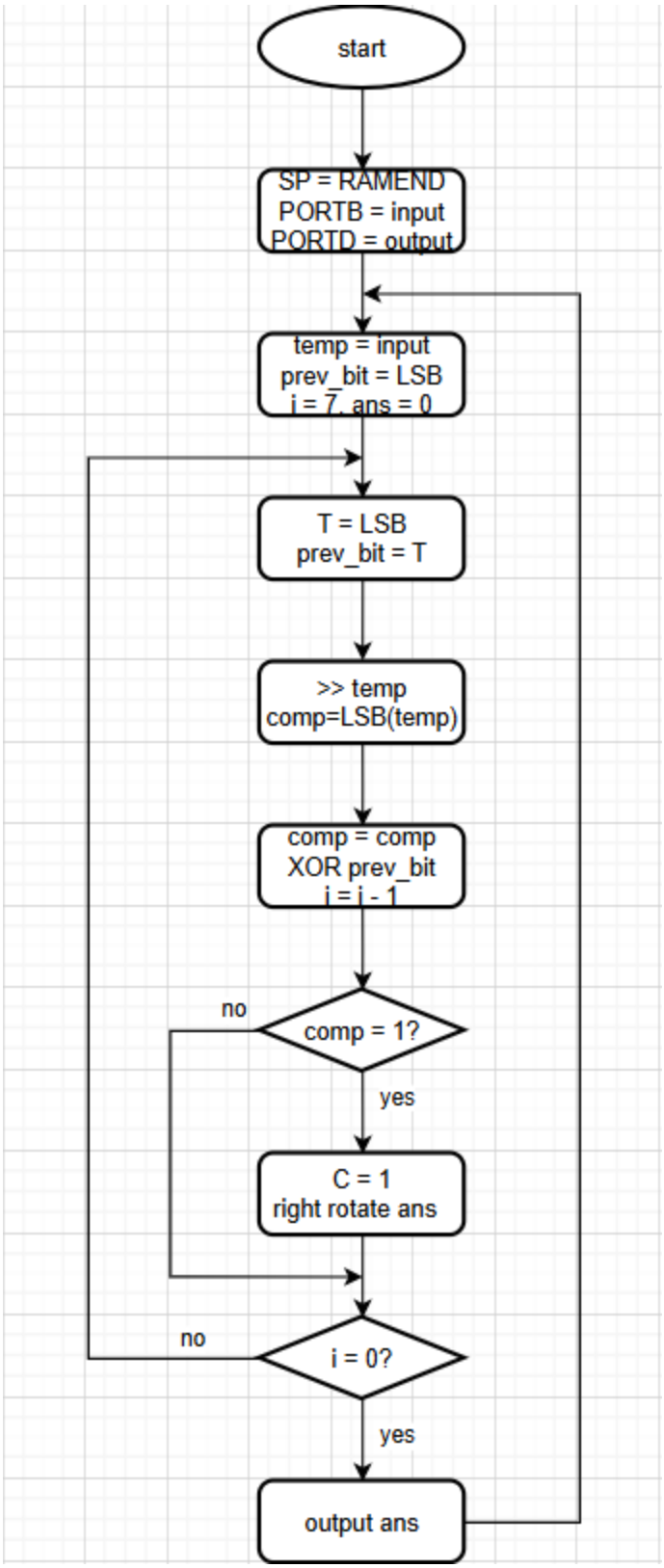
Για το παράδειγμα που δίνεται:



Το αποτέλεσμα είναι το αναμενόμενο:



Το διάγραμμα ροής της 2ης άσκησης είναι το ακόλουθο:


start
SP = RAMEND
PORTB = input
PORTD = output
temp = input
prev_bit = LSB
i = 7, ans = 0
T = LSB
prev_bit = T
>> temp
comp=LSB(temp)
comp = comp
XOR prev_bit
i = i - 1
no
comp = 1?
yes
C = 1
right rotate ans
no
i = 0?
yes
output ans

3η Άσκηση:

Ο κώδικας που υλοποιεί το λογικό κύκλωμα που φαίνεται στο σχήμα είναι ο ακόλουθος:

```
.include "m16def.inc"

.def temp = r16
.def ans = r17
.def A0 = r18
.def B0 = r19
.def A1 = r20
.def B1 = r21
.def A2 = r22
.def B2 = r23
.def A3 = r24
.def B3 = r25

.org 0x0000
    rjmp start

.org 0x002A ;start the program after the dedicated memory space for interrupts

start:
    ;stack pointer initialization
    ldi temp, low(RAMEND)
    out SPL, temp
    ldi temp, high(RAMEND)
    out SPH, temp

    ;I/O initialization
    clr temp
    out DDRA, temp
    ser temp
    out PORTA, temp ;pull-up resstance activation
    out DDRB, temp
```

```
init:
    in temp, PINA
    clr ans

    ;read the input appropriately
    mov B0, temp
    andi B0, 1
    lsr temp
    mov A0, temp
    andi A0, 1
    lsr temp
    mov B1, temp
    andi B1, 1
    lsr temp
    mov A1, temp
    andi A1, 1
    lsr temp
    mov B2, temp
    andi B2, 1
    lsr temp
    mov A2, temp
    andi A2, 1
    lsr temp
    mov B3, temp
    andi B3, 1
    lsr temp
    mov A3, temp
    andi A3, 1
    lsr temp
```

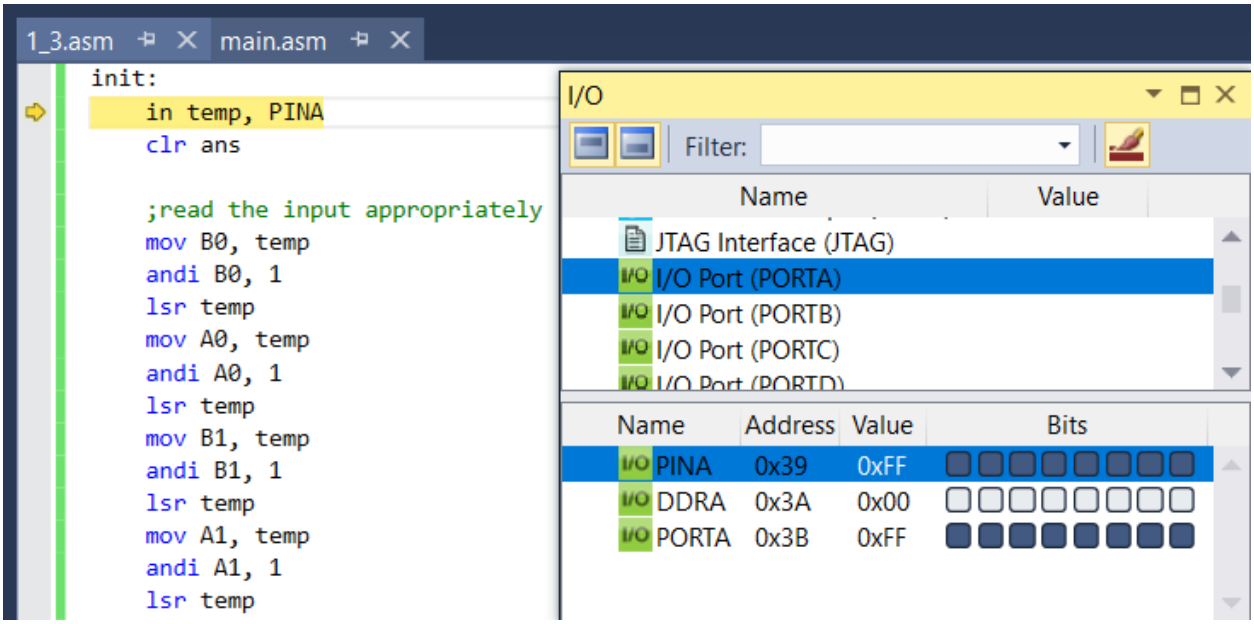
```
    ;1st layer logical functions
    and A3, B3
    and A2, B2
    or A1, B1
    or A0, B0

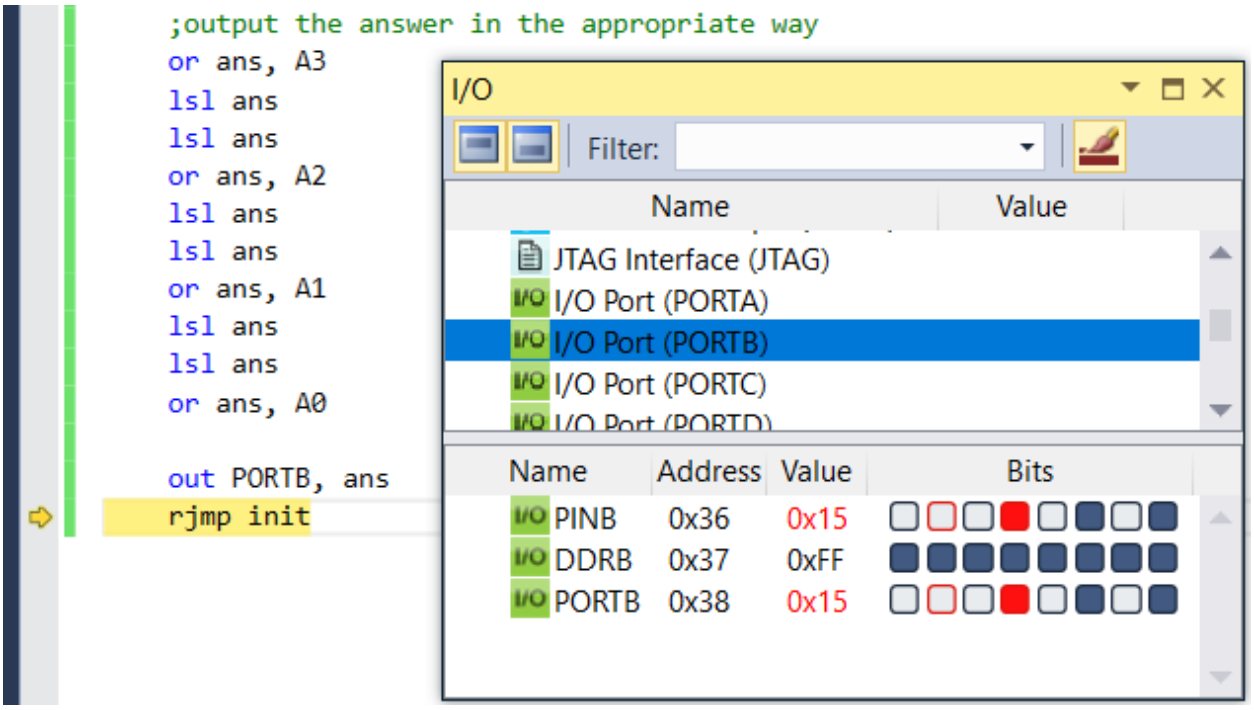
    ;2nd layer logical functions
    eor A3, A2
    and A1, A0

    ;output the answer in the appropriate way
    or ans, A3
    lsl ans
    lsl ans
    or ans, A2
    lsl ans
    lsl ans
    or ans, A1
    lsl ans
    lsl ans
    or ans, A0

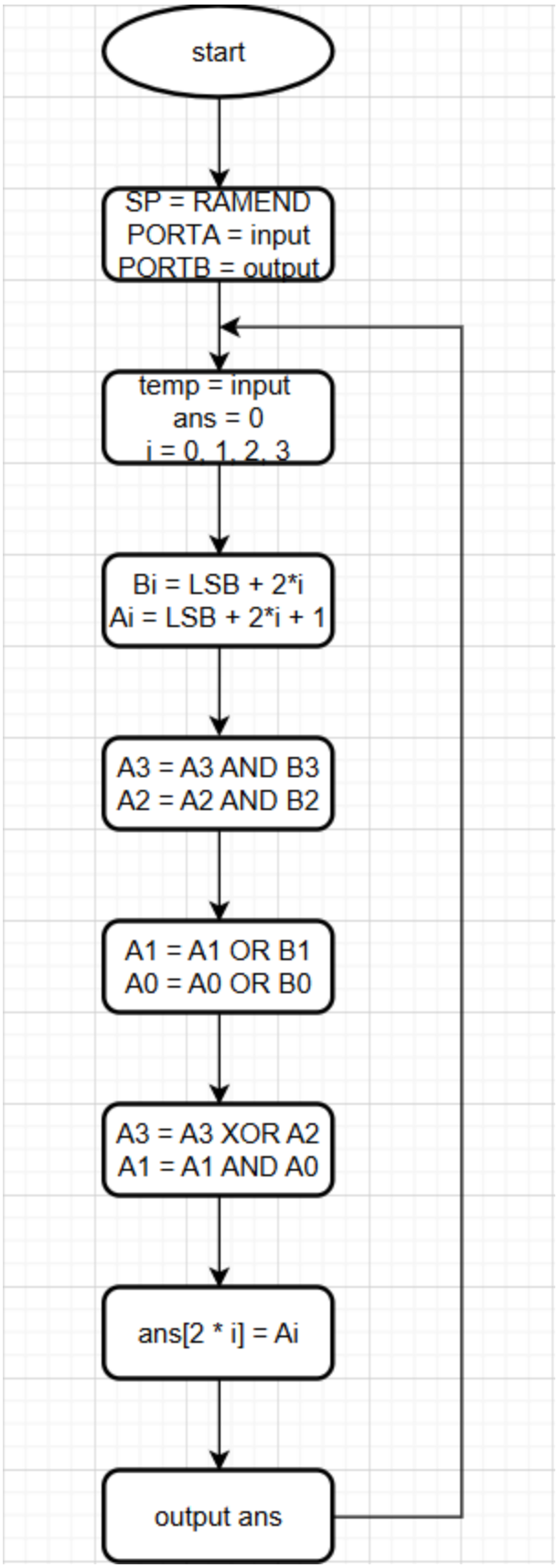
    out PORTB, ans
    rjmp init
```

Με είσοδο 11111111, το αποτέλεσμα είναι 0010101, το οποίο είναι το αναμενόμενο (όλες οι πύλες έχουν αποτέλεσμα 1, εκτός της XOR, τα αδιάφορα bit πάντα 0).





Το διάγραμμα ροής της 3ης άσκησης είναι το ακόλουθο:


start
SP = RAMEND
PORTA = input
PORTB = output
temp = input
ans = 0
i = 0, 1, 2, 3
Bi = LSB + 2*i
Ai = LSB + 2*i + 1
A3 = A3 AND B3
A2 = A2 AND B2
A1 = A1 OR B1
A0 = A0 OR B0
A3 = A3 XOR A2
A1 = A1 AND A0
ans[2 * i] = Ai
output ans

Ο κώδικας σε γλώσσα C είναι ο ακόλουθος:

```c
#include <avr/io.h>
#define byte uint8_t

int main(void){
    DDRA = 0x00; //PORTA is the input
    PORTA = 0xFF; //enable pull-up resistance
    DDRB = 0xFF; //PORTB is the output

    while(1){
        byte temp = PINA;
        byte ans = 0;

        //read the input appropriately
        byte B0 = temp & 1;
        temp >>= 1;
        byte A0 = temp & 1;
        temp >>= 1;
        byte B1 = temp & 1;
        temp >>= 1;
        byte A1 = temp & 1;
        temp >>= 1;
        byte B2 = temp & 1;
        temp >>= 1;
        byte A2 = temp & 1;
        temp >>= 1;
        byte B3 = temp & 1;
        temp >>= 1;
        byte A3 = temp & 1;
        temp >>= 1;

        //1st layer logical functions
        A3 &= B3;
        A2 &= B2;
        A1 |= B1;
        A0 |= B0;

        //2nd layer logical functions
        A3 ^= A2;
        A1 &= A0;

        //output the answer in the appropriate way
        ans |= A3;
        ans <<= 2;
        ans |= A2;
        ans <<= 2;
        ans |= A1;
        ans <<= 2;
        ans |= A0;

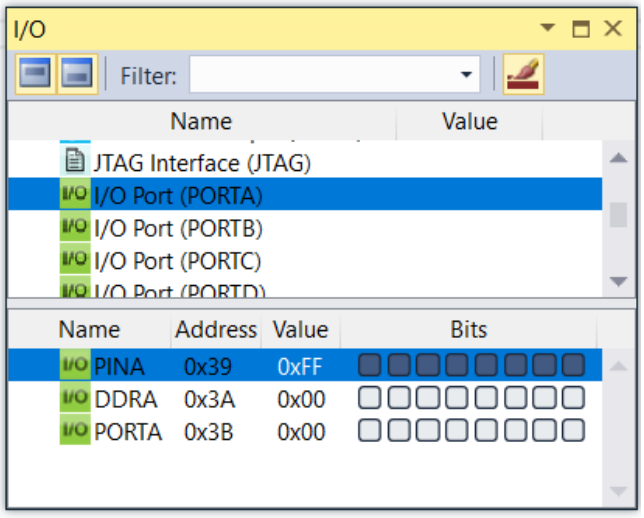
        PORTB = ans;
    }
    return 0;
}
```

Παρατηρούμε ότι για το ίδιο παράδειγμα,

```
int main(void){
    DDRA = 0x00; //PORTA is the input
    PORTA = 0xFF; //enable pull-up resistance
    DDRB = 0xFF; //PORTB is the output

    while(1){
        byte temp = PINA;
        byte ans = 0;

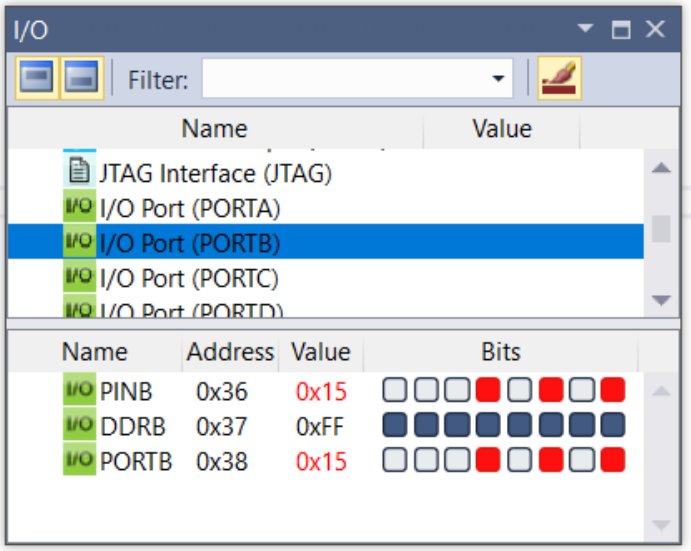
        //read the input appropriately
        byte B0 = temp & 1;
        temp >>= 1;
        byte A0 = temp & 1;
        temp >>= 1;
        byte B1 = temp & 1;
        temp >>= 1;
        byte A1 = temp & 1;
        temp >>= 1;
```



Τυπώνεται ξανά το ίδιο αναμενόμενο αποτέλεσμα.

```
    while(1){
        byte temp = PINA;
        byte ans = 0;

        //read the input appropriately
        byte B0 = temp & 1;
        temp >>= 1;
        byte A0 = temp & 1;
        temp >>= 1;
        byte B1 = temp & 1;
        temp >>= 1;
        byte A1 = temp & 1;
        temp >>= 1;
        byte B2 = temp & 1;
        temp >>= 1;
        byte A2 = temp & 1;
        temp >>= 1;
        byte B3 = temp & 1;
```



4η Άσκηση:

Ο κώδικας που παίρνει έναν αριθμό από τη θύρα εισόδου PORTA, τον μετατρέπει από δυαδική μορφή σε δεκαδική μορφή (το πλήθος των μονάδων στα 4 LSB και το πλήθος των δεκάδων στα 4 MSB) και τυπώνει το αποτέλεσμα στη θύρα εξόδου PORTC, είναι ο ακόλουθος. Για είσοδο >99, η έξοδος είναι πάντα 11111111.

```
.include "m16def.inc"

.def temp = r16
.def ans = r17

.org 0x0000
    rjmp start

.org 0x002A ;start the program after the dedicated memory space for interrupts

start:
    ;stack pointer initialization
    ldi temp, low(RAMEND)
    out SPL, temp
    ldi temp, high(RAMEND)
    out SPH, temp

    ;I/O initialization
    clr temp
    out DDRA, temp
    ser temp
    out PORTA, temp ;pull-up resstance activation
    out DDRC, temp

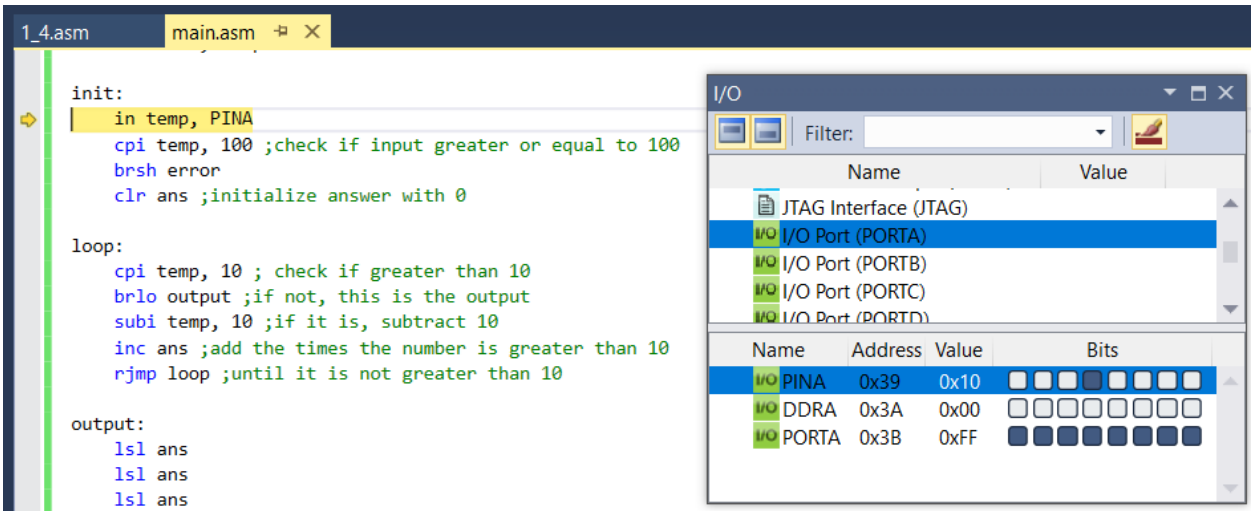
init:
    in temp, PINA
    cpi temp, 100 ;check if input greater or equal to 100
    brsh error
    clr ans ;initialize answer with 0
```

```
loop:
    cpi temp, 10 ; check if greater than 10
    brlo output ;if not, this is the output
    subi temp, 10 ;if it is, subtract 10
    inc ans ;add the times the number is greater than 10
    rjmp loop ;until it is not greater than 10

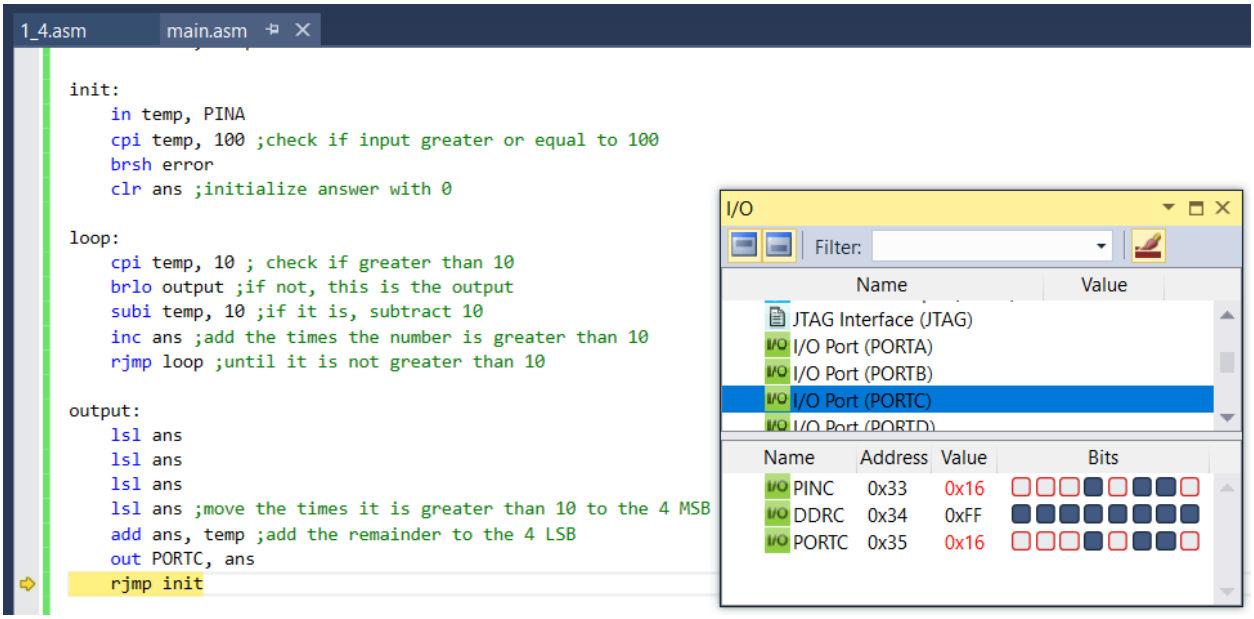
output:
    lsl ans
    lsl ans
    lsl ans
    lsl ans ;move the times it is greater than 10 to the 4 MSB
    add ans, temp ;add the remainder to the 4 LSB
    out PORTC, ans
    rjmp init

error:
    ser ans ;if it is, output is 11111111
    out PORTC, ans
    rjmp init
```

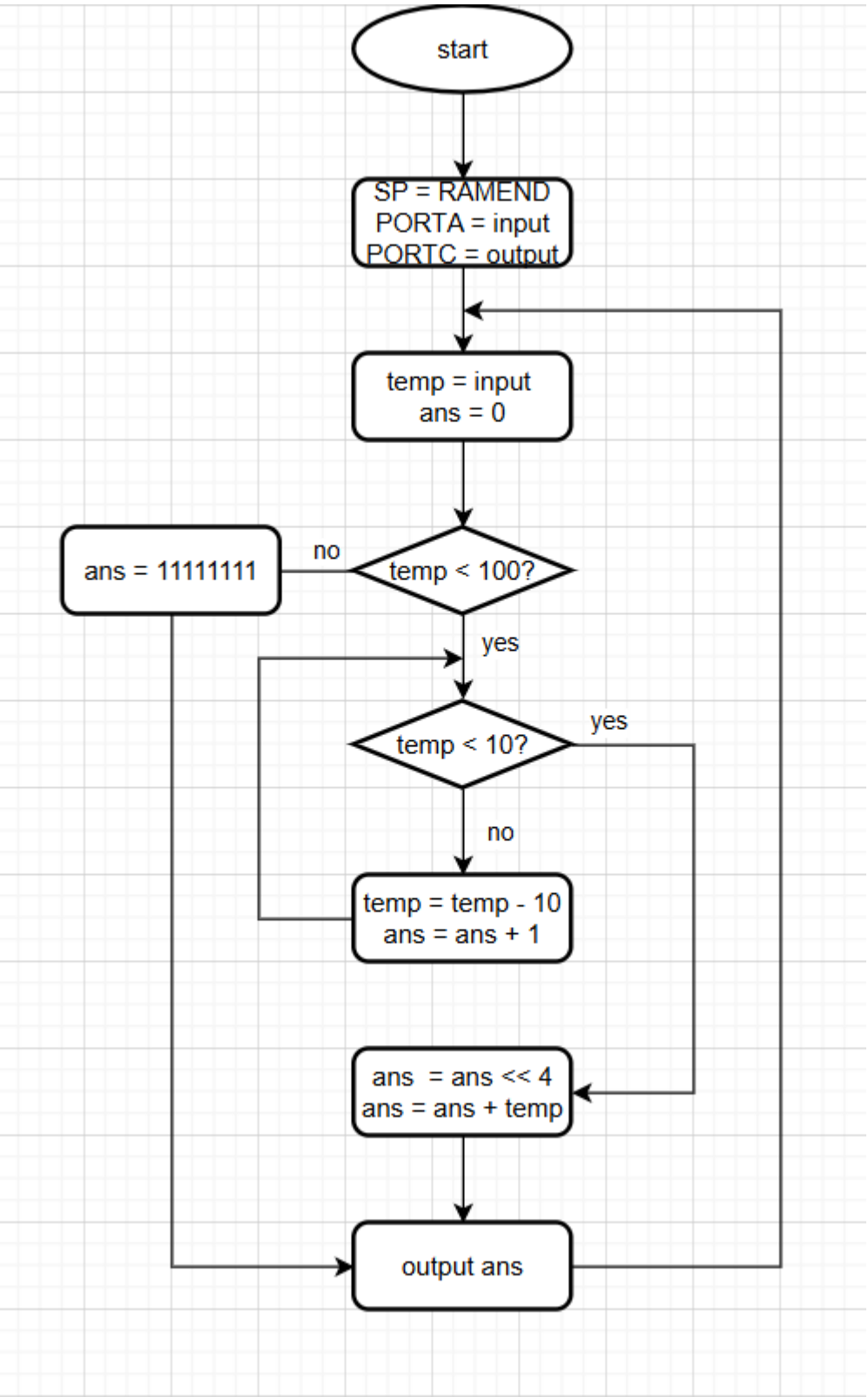
Για είσοδο 16 (00010000),



Το αποτέλεσμα είναι το αναμενόμενο (0001 0110, μία δεκάδα, έξι μονάδες).



Το διάγραμμα ροής της 4ης άσκησης είναι το ακόλουθο:



Ο κώδικας σε γλώσσα C είναι ο ακόλουθος:

```c
#include <avr/io.h>
#define byte uint8_t

int main(void){
    DDRA = 0x00; //PORTA is the input
    PORTA = 0xFF; //enable pull-up resistance
    DDRC = 0xFF; //PORTC is the output

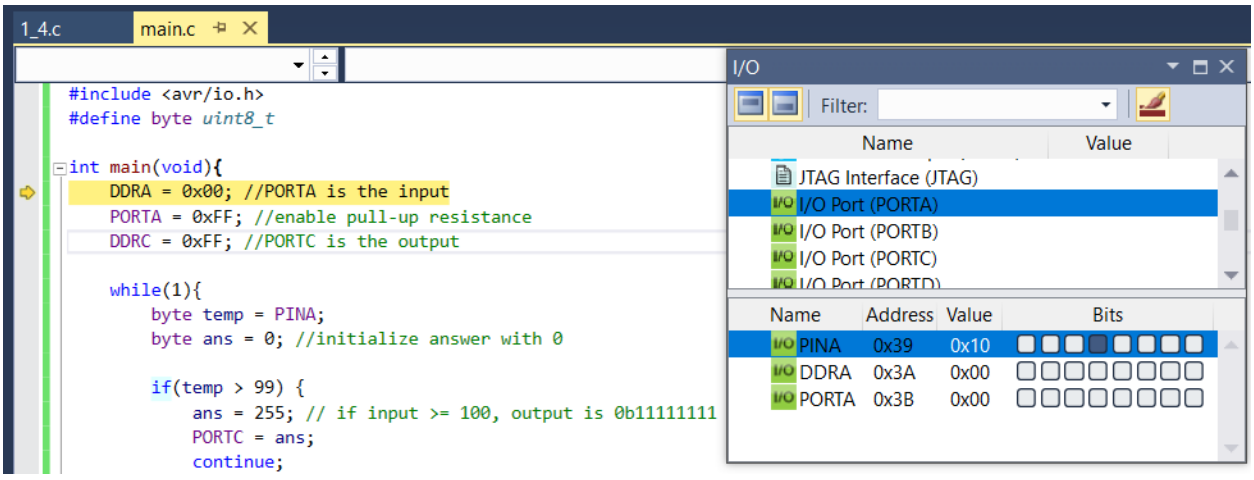
    while(1){
        byte temp = PINA;
        byte ans = 0; //initialize answer with 0

        if(temp > 99) {
            ans = 255; // if input >= 100, output is 0b11111111
            PORTC = ans;
            continue;
        }

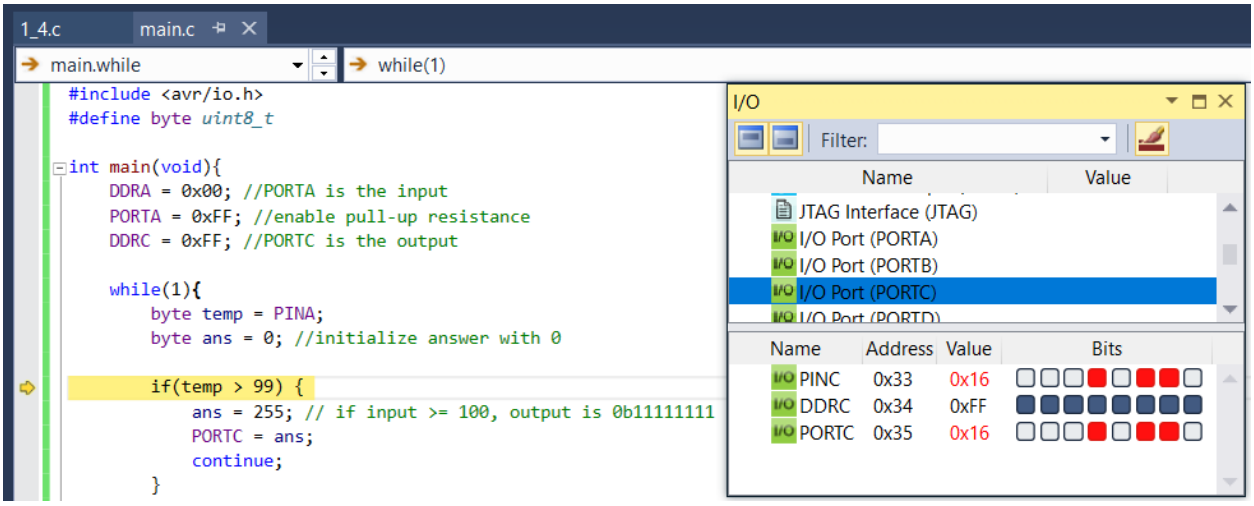
        while(temp >= 10){
            temp -= 10;
            ans +=1; //subtract all the times temp is greater than 9 and count them
        }

        ans <<= 4; //move them to the 4 MSB
        ans += temp; //add the remainder to the 4 LSB
        PORTC = ans;
    }
}
```

Παρατηρούμε όπως και πριν, για είσοδο 16,



Η έξοδος είναι η αναμενόμενη (0001 0110).



Δημήτριος Φανόγιαννης
03121902

1η Ομάδα Ασκήσεων
Συστήματα Μικροϋπολογιστών

Θεωρητικές Ασκήσεις

Στη Άσκηση

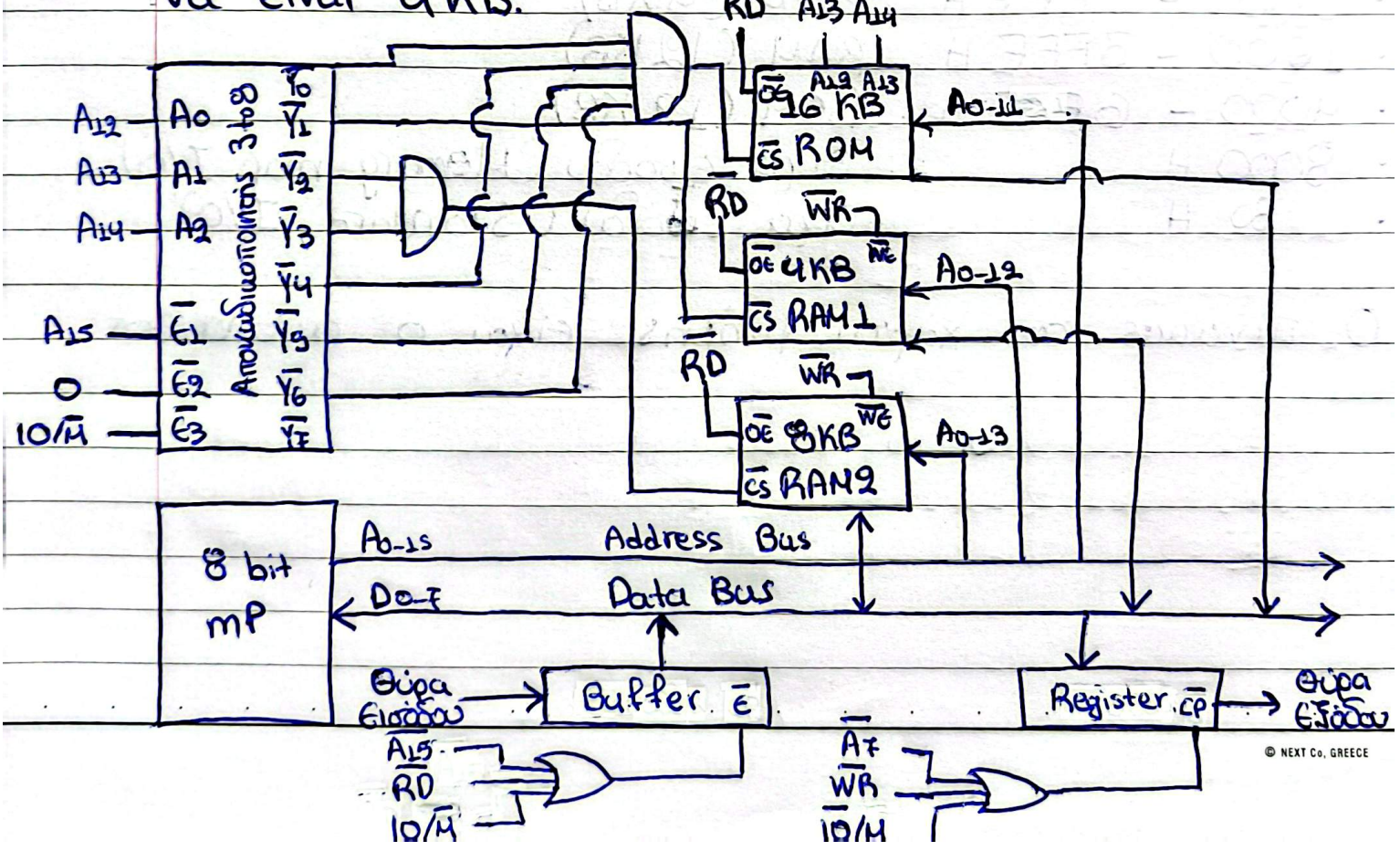
Έχουμε στη διάθεσή μας:
- 1 ROM των 16 KB
- 1 RAM των 4 KB
- 1 RAM των 8 KB
- καταχωρητές των 8 bit
- buffers των 8 bit
- κωδικοποιητές 3 σε 8
- βασικές λογικές πύλες

Ζητείται μΥ-2 8 bit επεξεργαστή με τον εξής χάρτη μνήμης
- 0000 - 0FFF H : ROM (4 KB)
- 1000 - 3FFF H : RAM (12 KB)
- 4000 - 6FFF H : ROM (12 KB)
- 8000 H : Θύρα εισόδου (Memory map I/O)
- 80 H : Θύρα εξόδου (Standard I/O)

Ο πίνακας του χάρτη μνήμης είναι ο ακόλουθος:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Address	Τύπος
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000	ROM
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0FFF	
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000	RAM 1
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1FFF	
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000	RAM 2
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFF	
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000	ROM
0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	6FFF	
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8000	Είσοδος

Εφόσον το μέγεθος της μικρότερης μονάδας μνήμης (RAM 1) είναι 4 KB, ~~θα χρησιμοποιήσω~~ και όλες οι άλλες μονάδες μνήμης είναι πολλαπλάσια των 4 KB, θα χρησιμοποιηθεί αποκωδικοποίηση μνημών των 4 KB, με την έξοδο του αποκωδικοποιητή 3 σε 8 να είναι 4 KB.



© NEXT Co, GREECE

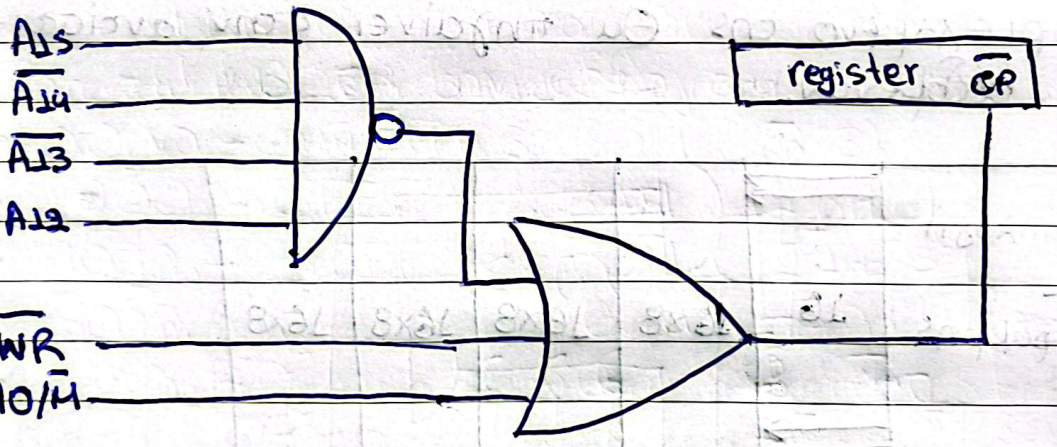
CamScanner

Στην περίπτωση όπου ο επεξεργαστής δε διαθέτει ξεχωριστές εντολές Ε/Ε και αντί της θύρας εξόδου 80 Η έχουμε την 9000 Η (Memory map I/O), οι αλλαγές στο σχεδιασμό του μΥ-2 είναι οι ακόλουθες:

Στον πίνακα χάρτη μνήμης, θα προστεθεί η ακόλουθη γραμμή:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Address	Τύπος
1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	9000	Έξοδος

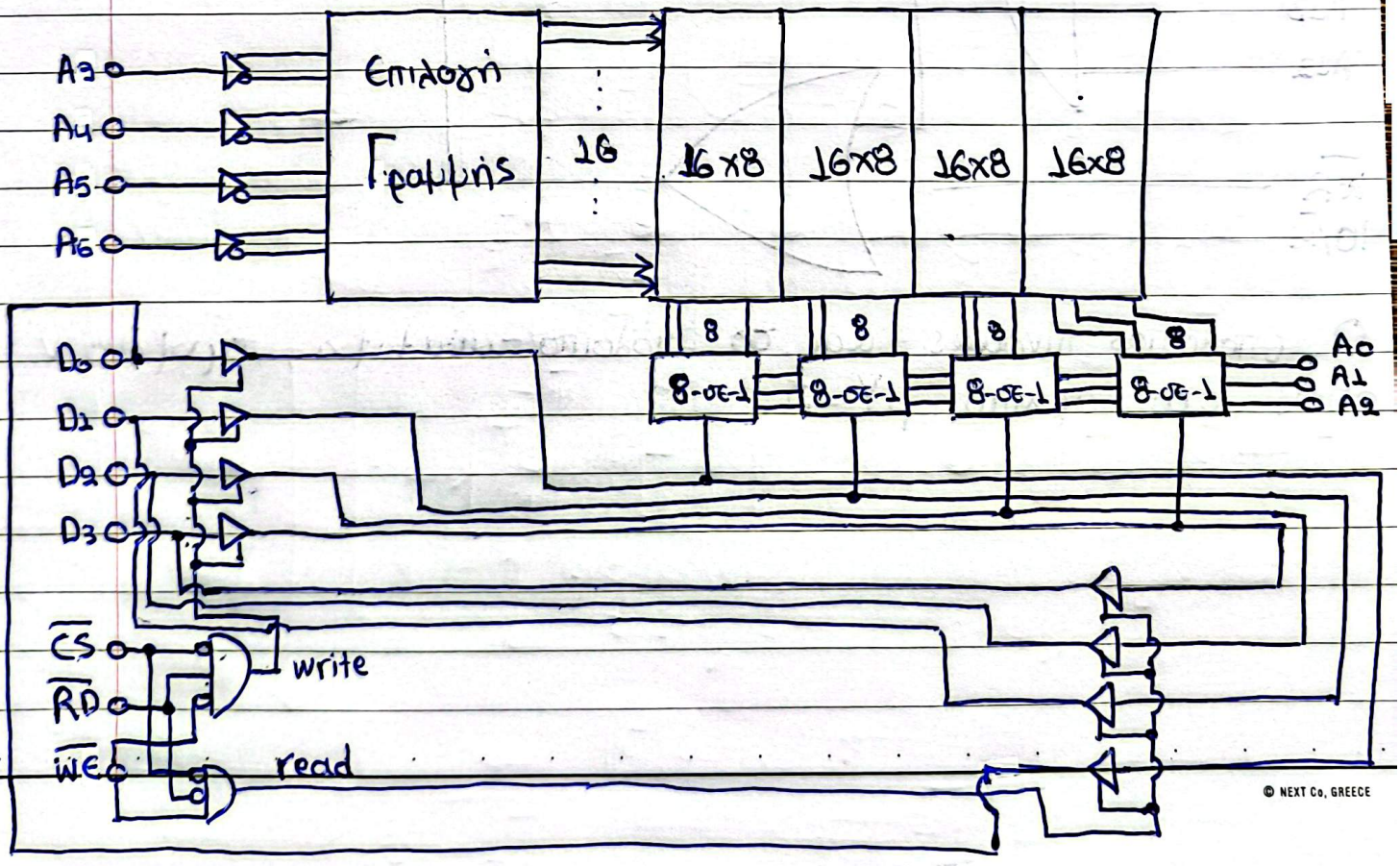
Στο κύκλωμα, η πύλη OR με τις εισόδους της, που ενεργοποιούν το σήμα $\overline{CP}$, θα αντικατασταθεί από την ακόλουθη πύλη με τις ακόλουθες εισόδους



Ο υπόλοιπος πίνακας και το υπόλοιπο κύκλωμα, παραμένουν όπως στο αρχικό μΥ-2.

5η Άσκηση

Η εσωτερική οργάνωση μιας μνήμης SRAM 128×4 bit απεικονίζεται παρακάτω. Εφόσον έχουμε 128 διευθύνσεις των 4 bit, χρειαζόμαστε συνολικά 7 γραμμές διεύθυνσης ($2^7 = 128$) και 4 γραμμές δεδομένων (4 bits). Για να γίνει η μνήμη όσο το δυνατό πιο τετραγωνισμένη, θα χωρίσαμε τις 7 γραμμές διευθύνσεων σε 4 για τις γραμμές και 3 για τις στήλες, δηλαδή συνολικά $2^4 = 16$ γραμμές και $2^3 = 8$ στήλες. Για να πιανοποιήσουμε την απαίτηση των 4 bit δεδομένων, θα χρησιμοποιήσουμε 4 τέτοια block μνημών (16×8), όπου η ίδια διεύθυνση θα επιλέγεται σε κάθε ένα από αυτά, και το περιεχόμενό της θα πηγαίνει στην αντίστοιχη γραμμή δεδομένων.



Στη συνέχεια θα γίνει ανάλυση της λειτουργίας ανάγνωσης και εγγραφής στη μνήμη αυτή. Έστω ότι διεύθυνση μνήμης που επιλέγεται για τις δύο παραπάνω λειτουργίες είναι η 1010101, δηλαδή $A_0 = A_2 = A_4 = A_6 = 1$ και $A_1 = A_3 = A_5 = 0$. Τα bits $A_3 - A_6$ μπαίνουν ως είσοδος στον αποκωδικοποιητή 4-σε-16, ο οποίος επιλέγει τις γραμμές στον πίνακα μνήμης. Η γραμμή που θα επιλεγεί είναι η 0101, δηλαδή η 5η γραμμή. Τα bits $A_0 - A_2$ μπαίνουν ως είσοδος στους 4 πολυπλέκτες 8-σε-1, οι οποίοι επιλέγουν τις στήλες στον πίνακα μνήμης. Η στήλη που θα επιλεγεί είναι η 101, δηλαδή η 5η στήλη, ίδια για κάθε block μνήμης και πολυπλέκτη.

Στην περίπτωση που επιθυμούμε ανάγνωση των στοιχείων της μνήμης, ενεργοποιούνται τα σήματα CS και RD, δηλαδή $\overline{CS} = 0$, $\overline{RD} = 0$, $\overline{WE} = 1$, με αποτέλεσμα μόνο η πύλη AND read να ενεργοποιείται, και μαζί με αυτή οι buffers που επιτρέπουν τη ροή από τη μνήμη στις γραμμές των δεδομένων

Στην περίπτωση που επιθυμούμε εγγραφή στοιχείων στη μνήμη, ενεργοποιούνται τα σήματα CS και WE, δηλαδή $\overline{CS} = 0$, $\overline{RD} = 1$, $\overline{WE} = 0$, με αποτέλεσμα μόνο η πύλη AND write να ενεργοποιείται, και μαζί με αυτή οι buffers που επιτρέπουν τη ροή από τις γραμμές των δεδομένων στη μνήμη.

6η Άσκηση

Έχουμε στη διάθεσή μας:
- 1 ROM των 2 KB
- 1 ROM των 4 KB
- 1 SRAM των 2 KB
- 1 SRAM των 8 KB

Ζητείται σύστημα μνήμης 8-bit επεξεργαστού με τον ακόλουθο χάρτη μνήμης:
- 1000H - 17FFH: ROM_1 (2 KB)
- 1800H - 27FFH: ROM_2 (4 KB)
- 2800H - 2FFFH: $SRAM_1$ (2 KB)
- 3000H - 4FFFH: $SRAM_2$ (8 KB)

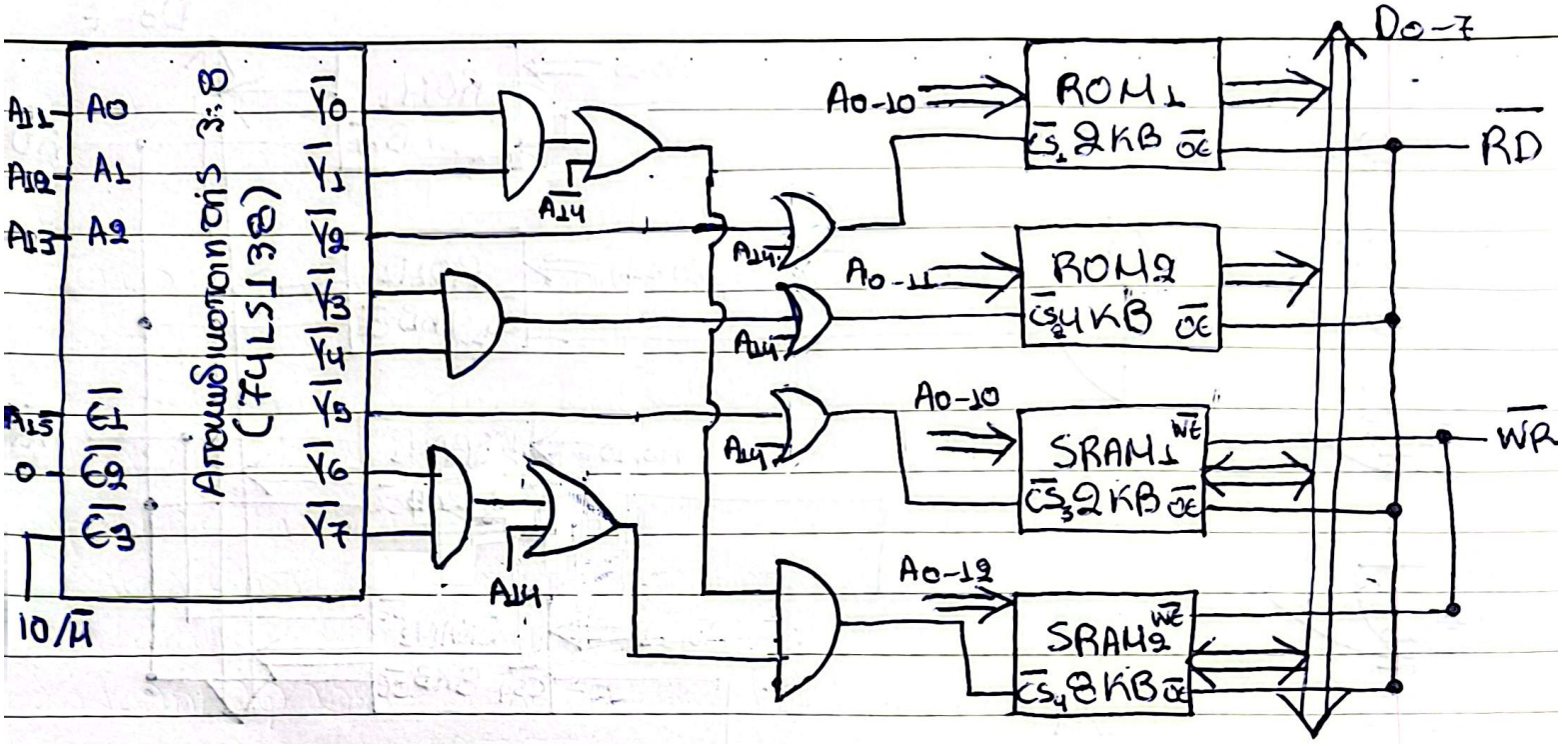
Ο πίνακας του χάρτη μνήμης είναι ο ακόλουθος:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Address	Τύπος
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000	ROM_1
0	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	17FF	
0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1800	ROM_2
0	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	27FF	
0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	2800	$SRAM_1$
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	2FFF	
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	3000	$SRAM_2$
0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	4FFF	

α) Η υλοποίηση του παραπάνω συστήματος μνήμης με έναν αποκωδικοποιητή 3:8 και δοχιμές πύλες είναι η ακόλουθη

© NEXT Co, GREECE

CamScanner



Στην παραπάνω υλοποίηση χρησιμοποιήθηκε σελιδοποίηση των 2KB, εφόσον αυτό είναι το μέγεθος της μικρότερης μονάδας μνήμης και όλες οι άλλες μονάδες μνήμης είναι πολλαπλάσια των 2KB

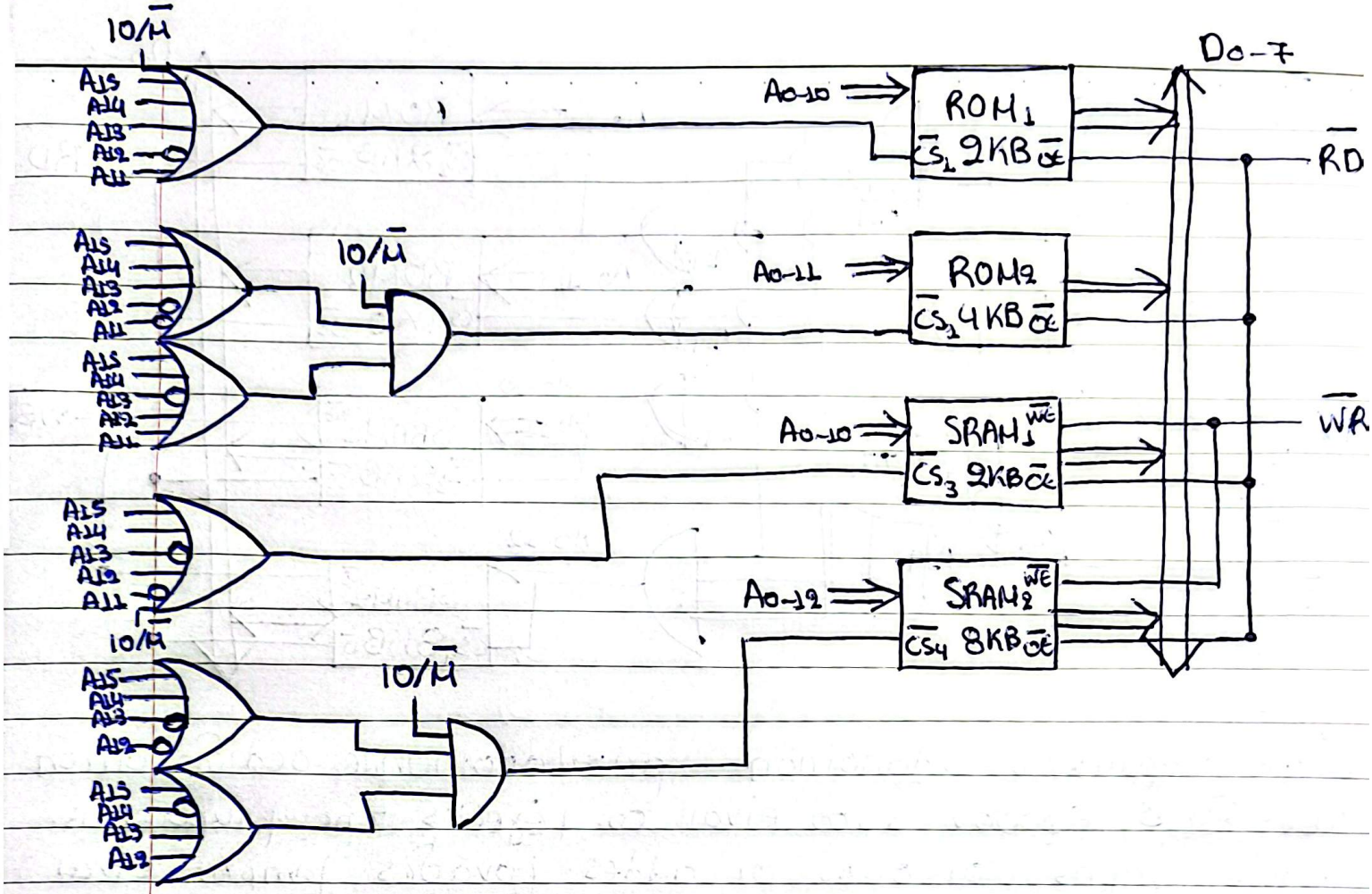
β) Η υλοποίηση του παραπάνω συστήματος μνήμης μόνο με λογικές πύλες είναι η ακόλουθη. Τα σήματα Chip Select της κάθε μονάδας μνήμης είναι τα εξής:

$\overline{CS_1} = A_{15} + A_{14} + A_{13} + \overline{A_{12}} + A_{11}$ (ROM1)

$\overline{CS_2} = (A_{15} + A_{14} + A_{13} + \overline{A_{12}} + \overline{A_{11}}) \cdot (\overline{A_{15}} + A_{14} + \overline{A_{13}} + A_{12} + A_{11})$ (ROM2)

$\overline{CS_3} = A_{15} + A_{14} + \overline{A_{13}} + A_{12} + A_{11}$ (SRAM1)

$\overline{CS_4} = (A_{15} + A_{14} + \overline{A_{13}} + \overline{A_{12}} + A_{11}) \cdot (\overline{A_{15}} + A_{14} + \overline{A_{13}} + \overline{A_{12}} + \overline{A_{11}}) \cdot$
$\cdot (A_{15} + A_{14} + \overline{A_{13}} + \overline{A_{12}} + A_{11}) \cdot (\overline{A_{15}} + A_{14} + A_{13} + A_{12} + A_{11}) =$
$= (A_{15} + A_{14} + A_{13} + A_{12}) \cdot (\overline{A_{15}} + A_{14} + A_{13} + A_{12})$



© NEXT Co, GREECE

CamScanner